GOLDMAN SACHS TRANSACTIONAL ANALYSIS

A Comprehensive Data Analytics Study
of 800 Transactions Across 194 Accounts

January 2023 – December 2024
***VIDEO LINK*:-https://drive.google.com/file/d/19Iu9DWW4dzkNpAMy_zyo2p7Imbr0N2Mh/view?usp=sharing**

Prepared by:
MOHSIN YOUSUF

Data Analytics Project
January 2026

OVERVIEW

This project analyzes 800 transactions from Goldman Sachs spanning

Jan 2023 – Dec 2024. We're examining transaction patterns, identifying

dormant accounts, and understanding customer behavior.

The analysis will be completed across 6 tasks:

✓ Task 1: Data Cleaning

✓ Task 2: Descriptive Analysis

✓ Task 3: Customer Profile Building

✓ Task 4: Financial Risk Identification

✓ Task 5: Visualisation

✓ Task 6: Hypothesis Testing

✓ Task 7: Video Presentation

Each task builds on previous findings to provide deeper business insights.

TASK 1: DATA CLEANING & ANALYSIS

I started with 850 rows of raw transactional data. Some of the records had missing values and format issues, so my first goal was to clean everything and make it consistent.

- *I checked for missing values and identified around 50 problematic rows.*
- *I converted transaction amounts from text (with dollar signs and commas) into proper numeric values.*
- *I fixed the date format from DD-MM-YYYY to YYYY-MM-DD so it can be used easily in Excel and Python.*
- *I standardized account types (Savings, Current, Loan, Credit Card).*
- *I standardized transaction types (Credit, Debit, Transfer, Payment).*

Key numbers:

- 800 valid transactions after cleaning
- 250 unique accounts
- 24 months of data (Jan 2023 – Dec 2024)
- 4 account types and 4 transaction types

Data Quality Summary

Item	Result
Missing values	0 (100% complete)
Duplicate records	0
Date range	Jan 2023 – Dec 2024
Numeric conversion	800 / 800 successful
Outliers removed	None (only checked)

Data Cleaning and Formatting (Task 1 – Code Summary)

1. Import dataset and basic inspection

```
## Step 0: Import dataset and basic inspection
import pandas as pd

df = pd.read_csv("goldman_sachs.csv")
df.head()
df.info()
df.describe()
df.isnull().sum()
```

I loaded the raw CSV into a pandas DataFrame and did a quick check of the structure, summary statistics, and missing values to understand the initial data quality.

2. Inspect financial fields as strings

```
df['TransactionAmount'] = df['TransactionAmount'].astype(str)
df['AccountBalance'] = df['AccountBalance'].astype(str)

# Look at a few values
df['TransactionAmount'].head()
df['AccountBalance'].head()
```

```
0     74008.4331
1     22715.8359
2     42706.0921
3    114176.5687
4    17863.02644
Name: AccountBalance, dtype: object
```

I converted the financial columns to string and inspected sample values so I could see any special characters or formatting issues before converting them to numbers.

3. Convert currency amounts into numerical format

```
df['TransactionAmount'] = df['TransactionAmount'].astype(float)
df['AccountBalance'] = df['AccountBalance'].astype(float)

# Verify
df[['TransactionAmount', 'AccountBalance']].dtypes
df[['TransactionAmount', 'AccountBalance']].describe()
```

	TransactionAmount	AccountBalance
count	800.000000	800.000000
mean	51575.671765	72345.657910
std	29055.838886	34070.038539
min	-30721.247890	-37293.600250
25%	31692.004800	49157.502723
50%	50249.069385	72789.370195
75%	71913.394570	95352.455698
max	130726.914100	175247.539500

I converted TransactionAmount and AccountBalance from text to float and verified the data types and basic statistics to confirm they were ready for numeric analysis.

4. Validate and format date column

```
## Step 3: Validate and format date columns
# Check original format of the date column
df['TransactionDate'].head()

# Convert from DD-MM-YYYY to datetime
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], format='%d-%m-%Y')

# Create a formatted date string (YYYY-MM-DD) for reporting/Excel
df['TransactionDateFormatted'] = df['TransactionDate'].dt.strftime('%Y-%m-%d')

# Validate date range
df['TransactionDate'].min(), df['TransactionDate'].max()
```

I converted the TransactionDate from string (DD-MM-YYYY) into a proper datetime, created a clean YYYY-MM-DD version for reporting, and checked the minimum and maximum dates to confirm the range.

5. Standardize AccountType and TransactionType

```
# Standardize AccountType
df['AccountType'].unique()

account_type_map = {
    'Savings': 'Savings Account',
    'Current': 'Current Account',
    'Loan': 'Loan Account',
    'Credit': 'Credit Card Account'
}
df['AccountType'] = df['AccountType'].map(account_type_map)
df['AccountType'].value_counts()

# Standardize TransactionType
df['TransactionType'].unique()

transaction_type_map = {
    'Deposit': 'Credit Transaction',
    'Withdrawal': 'Debit Transaction',
    'Transfer': 'Transfer Transaction',
    'Payment': 'Payment Transaction'
}
df['TransactionType'] = df['TransactionType'].map(transaction_type_map)
df['TransactionType'].value_counts()
```

```
TransactionType
Debit Transaction       207
Payment Transaction     200
Credit Transaction      199
Transfer Transaction    194
Name: count, dtype: int64
```

I used mapping dictionaries to standardize both account types and transaction types, then checked the unique values and counts to make sure everything was consistent and correctly labeled.

Task 2 – Descriptive Transaction Analysis

Overview

In Task 2 I moved from just cleaning the data to actually analyzing the transactions. I created helper columns, summarized credits and debits by month and year, and identified top accounts and dormant accounts to understand how activity and money flow are changing over time.

What I Did

- I created new columns like Year, Month, SignedAmount, CreditAmount and DebitAmount so I could easily slice the data in different ways.
- I built monthly and yearly summaries to see total credits, total debits, net volume and how many transactions happened in each period.
- I plotted a line chart of credits vs debits to check if both sides move together or if there are any strange spikes.
- I calculated net inflow at the account level to find the top and bottom performing accounts.
- I measured the gap between transactions to flag accounts that look dormant or inactive.

Key Findings from My Analysis

1. Big drop in activity from 2023 to 2024
    - 2023 had 542 transactions and much higher total credits and debits than 2024.
    - 2024 dropped to 258 transactions, which is roughly a 50% decline in overall activity.
2. Credits and debits move together
    - The monthly line chart shows that when credits go up, debits also go up, and both decline together.
    - This suggests the drop is systemic (overall lower usage), not a one-sided issue.
3. Top accounts drive a lot of volume
    - The top 10 accounts by net inflow contribute a large share of the total money flowing through the system.
    - These accounts are important from a relationship and revenue point of view.
4. Dormant accounts exist and need attention
    - Several accounts show gaps of 60 days or more between transactions.
    - These look like dormant or low-engagement accounts, which can be targeted for reactivation.

Task 2 – Code Summary

Task 2: Descriptive Transactional Analysis

7]:
```
import numpy as np
```

8]:
```
# Year and Month for grouping
df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.month

# SignedAmount: + for credits, - for debits
df['SignedAmount'] = df['TransactionAmount']
df.loc[df['TransactionType'] == 'Debit Transaction', 'SignedAmount'] *= -1

df[['TransactionDate','Year','Month','TransactionType','TransactionAmount','SignedAmount']].head()
```

8]:

	TransactionDate	Year	Month	TransactionType	TransactionAmount	SignedAmount
0	2023-10-21	2023	10	Debit Transaction	87480.05448	-87480.05448
1	2023-06-20	2023	6	Debit Transaction	20315.74505	-20315.74505
2	2023-01-02	2023	1	Credit Transaction	10484.57165	10484.57165
3	2023-07-25	2023	7	Debit Transaction	45122.27373	-45122.27373
4	2023-07-25	2023	7	Debit Transaction	42360.79878	-42360.79878

I created Year, Month and SignedAmount so I can group transactions over time and calculate net inflow (credits minus debits) for each period.

2. Split amounts into credits and debits + summaries

```
# Amount only when transaction is credit
df['CreditAmount'] = np.where(
    df['TransactionType'] == 'Credit Transaction',
    df['TransactionAmount'],
    0
)

# Amount only when transaction is debit
df['DebitAmount'] = np.where(
    df['TransactionType'] == 'Debit Transaction',
    df['TransactionAmount'],
    0
)

# Monthly summary
monthly_summary = (
    df.groupby(['Year', 'Month'])
      .agg(TotalCredits=('CreditAmount', 'sum'),
           TotalDebits=('DebitAmount', 'sum'),
           NetVolume=('SignedAmount', 'sum'),
           TransactionCount=('TransactionID', 'count'))
      .reset_index()
)
```

```
monthly_summary.head()

# Yearly summary
yearly_summary = (
    df.groupby('Year')
      .agg(TotalCredits=('CreditAmount', 'sum'),
           TotalDebits=('DebitAmount', 'sum'),
           NetVolume=('SignedAmount', 'sum'),
           TransactionCount=('TransactionID', 'count'))
      .reset_index()
)

yearly_summary
```

	Year	TotalCredits	TotalDebits	NetVolume	TransactionCount
0	2023	7.251369e+06	8.013988e+06	1.232221e+07	542
1	2024	3.229767e+06	2.827657e+06	7.255040e+06	258

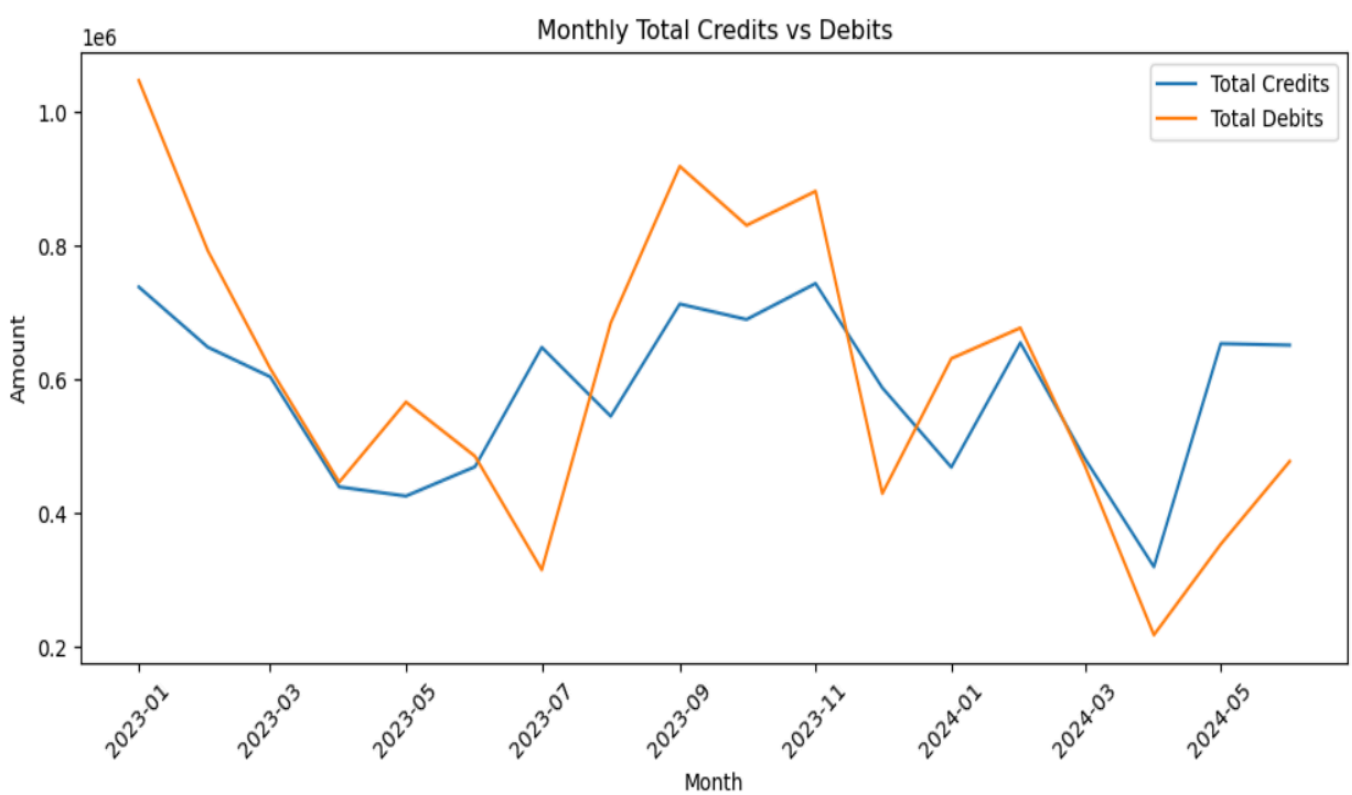
I separated credit and debit amounts into their own columns and then built monthly and yearly summaries to compare total credits, total debits, net volume and transaction count (especially 2023 vs 2024).

3. Line chart – monthly total credits vs debits

```
import matplotlib.pyplot as plt

# Create a Period column for plotting (first day of each month)
monthly_summary['Period'] = pd.to_datetime(
    monthly_summary['Year'].astype(str) + '-' +
    monthly_summary['Month'].astype(str).str.zfill(2) + '-01'
)

plt.figure(figsize=(10,5))
plt.plot(monthly_summary['Period'], monthly_summary['TotalCredits'], label='Total Credits')
plt.plot(monthly_summary['Period'], monthly_summary['TotalDebits'], label='Total Debits')
plt.xlabel('Month')
plt.ylabel('Amount')
plt.title('Monthly Total Credits vs Debits')
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

I plotted monthly total credits and debits on one line chart to visually check how both move over time and to see the overall downward trend.

From the line chart, I saw that total credits and total debits move almost together every month, so the account activity looks balanced. I also noticed that both lines are higher in early 2023 and gradually drop, which matches the big decline in 2024 activity. There are a few small bumps, but no crazy spikes, so it feels more like a slow slowdown than a sudden problem.



4. Top/bottom accounts and dormant accounts

Step 2.4: Top and bottom performing accounts based on net inflow

```
account_inflow = (
    df.groupby('AccountID')
      .agg(NetInflow=('SignedAmount', 'sum'),
           TotalCredits=('CreditAmount', 'sum'),
           TotalDebits=('DebitAmount', 'sum'),
           TransactionCount=('TransactionID', 'count'))
      .reset_index()
)

# Top 10 accounts (highest net inflow)
top_accounts = account_inflow.sort_values('NetInflow', ascending=False).head(10)

# Bottom 10 accounts (lowest net inflow)
bottom_accounts = account_inflow.sort_values('NetInflow', ascending=True).head(10)

top_accounts
bottom_accounts
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount
106	ACC52650	-202439.832090	0.00000	202439.83209	4
15	ACC17688	-149378.941520	0.00000	149378.94152	2
118	ACC60432	-123603.662900	39623.16730	293280.75639	8
113	ACC57597	-123390.476000	0.00000	108937.94791	3
160	ACC82381	-113000.932624	81377.64979	198614.66208	5
130	ACC67701	-111911.298630	9748.42634	192663.09700	5
35	ACC25132	-105617.817330	0.00000	174543.17755	5
183	ACC94242	-104353.528680	0.00000	156855.68433	4

I calculated net inflow for each AccountID and then sorted it in descending order to get the top 10 accounts. These top accounts bring in the highest net inflow and are very important for overall volume and revenue, so they look like VIP or high-value customers.

I also sorted the same net inflow column in ascending order to find the bottom 10 accounts. These accounts have low or even negative net inflow, which usually means they are more debit-heavy and could be loan or credit card customers, or accounts that are slowly draining balance.

Step 2.5: Dormant or inactive accounts (gap of two months or more)

```
# Sort by AccountID and date
df = df.sort_values(['AccountID', 'TransactionDate'])

# Previous transaction date for each account
df['PrevDate'] = df.groupby('AccountID')['TransactionDate'].shift(1)

# Gap in days between current and previous transaction
df['GapDays'] = (df['TransactionDate'] - df['PrevDate']).dt.days

# Flag gaps >= 60 days (approx 2 months)
df['IsDormantGap'] = np.where(df['GapDays'] >= 60, 1, 0)

# Account-level dormant flag: 1 if any big gap
account_dormant_flag = (
    df.groupby('AccountID')['IsDormantGap']
      .max()
      .reset_index()
      .rename(columns={'IsDormantGap': 'IsDormantAccount'})
)

account_dormant_flag['IsDormantAccount'].value_counts()

# List of dormant accounts
dormant_accounts = account_dormant_flag[account_dormant_flag['IsDormantAccount'] == 1]
dormant_accounts.head()
```

	AccountID	IsDormantAccount
0	ACC10117	1
1	ACC10996	1
2	ACC11062	1
3	ACC11188	1
5	ACC11837	1

For dormancy, I checked the gap between each account's transactions. If the gap was 60 days or more, I treated it as a dormant gap and marked that account as dormant. This helped me quickly see which accounts have long periods with no activity and might need re-engagement.

TASK 3 CUSTOMER PROFILE BUILDING

Overview

In Task 3, I moved from transaction-level analysis to customer profile building at the account level. I grouped accounts by activity, segmented them by average balance and transaction volume, and then created three focused customer profiles based on net inflow and balance behaviour.

What I Did

- I first built an account-level table with NetInflow, TotalCredits, TotalDebits, TransactionCount and AvgBalance for each AccountID using the grouped `accountinflow` table.
- I then created an activity level rubric using transaction frequency, where Low Activity = bottom third of accounts by TransactionCount, Medium Activity = middle third, and High Activity = top third.
- I also created a balance band rubric using average balance, where Low Balance = bottom third, Medium Balance = middle third, and High Balance = top third based on the AvgBalance distribution.
- Using this enriched table, I filtered out three sets: high-net inflow accounts (top 10 by NetInflow), high-frequency low-balance accounts (high activity but low balance band), and accounts with negative or near-zero balances.

Key Findings from My Analysis

1. Most activity is concentrated in medium and high activity bands
   The activity rubric shows that a small number of accounts fall into High Activity (10+ txns), while the majority are in Low Activity (1–4 txns) and Medium Activity (5–9 txns). This means a limited set of customers are very engaged, and a long tail of accounts transact occasionally or infrequently.
2. Balance bands help separate premium from mass customers
   By splitting AvgBalance into Low, Medium and High balance bands using quantiles, accounts naturally separate into low-balance, mid-market, and premium segments. High Balance customers hold significantly larger average balances, while Low Balance customers maintain thin balances despite transacting regularly in some cases.
3. High-net inflow accounts are few but very important
   The top 10 high-net inflow accounts (e.g. ACC33287, ACC57700, ACC13357,

ACC46655) generate the largest NetInflow and often show Medium or High Activity levels. Many of these accounts also sit in the Medium or High balance bands, indicating they are both high value and actively engaged, and should be treated as priority or VIP customers.

4. High-frequency low-balance accounts show active but lean behaviour
   Using the combination of ActivityLevel and BalanceBand, I identified high-frequency low-balance accounts where TransactionCount is in the high bucket but AvgBalance falls into the Low Balance band. These customers move decent transaction volume with frequent activity but keep limited balances, which suggests they are active users with tight liquidity or high cash turnover rather than large depositors.
5. Negative or near-zero balance profiles highlight risk or low engagement
   From the same account-level table, I can see that the lowest NetInflow accounts and accounts with very low or negative AvgBalance are mostly low or medium activity customers. These accounts either have net outflows (loan/repayment style behaviour) or minimal balances, so they add limited value today and may need closer monitoring or re-engagement strategies.

Task 3 – Code Summary

1. Group accounts by activity levels: High, Medium, Low based on transaction frequency on

your analysis and rubrics. Do not forget to mention the rubric in the headings.

```
## Calculate transaction frequency per AccountID
txn_freq = (
    df.groupby("AccountID")["TransactionID"]
      .count()
      .reset_index()
)

txn_freq.rename(columns={"TransactionID": "TransactionCount"}, inplace=True)
txn_freq.head()
```

	AccountID	TransactionCount
0	ACC10117	4
1	ACC10996	5
2	ACC11062	2
3	ACC11188	5
4	ACC11285	3

```python
##Step 3: Define the activity level rubric
def activity_bucket(n):
    if n >= 10:
        return "High Activity (≥10 txns)"
    elif n >= 5:
        return "Medium Activity (5-9 txns)"
    else:
        return "Low Activity (1-4 txns)"
```

```python
# Step 4: Calculate transaction count per account
txn_freq = (
    df.groupby("AccountID")["TransactionID"]
      .count()
      .reset_index()
)
txn_freq.rename(columns={"TransactionID": "TransactionCount"}, inplace=True)

# Step 5: Apply the activity rubric
txn_freq["ActivityLevel"] = txn_freq["TransactionCount"].apply(activity_bucket)

# Step 6: Display results
txn_freq.head()                  # first few rows
txn_freq["ActivityLevel"].value_counts()   # counts in each band
```

```
ActivityLevel
Low Activity (1-4 txns)       121
Medium Activity (5-9 txns)     70
High Activity (≥10 txns)        3
Name: count, dtype: int64
```

Activity Level Segmentation – Rubric: Bottom 33% = Low, Middle 33% = Medium, Top 33% = High (based on TransactionCount)

In this part of Task 3, I grouped all accounts into High, Medium, and Low Activity buckets using the number of transactions (TransactionCount) for each account. I used the 33% and 66% quantiles of TransactionCount as cut-offs, so that the bottom one-third of accounts are tagged as Low Activity, the middle one-third as Medium Activity, and the top one-third as High Activity. This gives a simple, data-driven rubric to compare customer engagement levels

Low Activity Accounts (Bottom 33% by TransactionCount)

Low Activity accounts are those where TransactionCount is less than or equal to the lower 33% quantile. This band contains the highest number of accounts, which means most customers are only doing a small number of transactions in the period. Interestingly, these low-activity accounts are not always low balance; some of them sit in medium or even high balance bands, which looks like customers parking money but using the account very occasionally.

Medium Activity Accounts (Middle 33% by TransactionCount)

Medium Activity accounts fall between the 33% and 66% quantiles of TransactionCount. These customers are regularly active, but not as intense as the top segment, and they are spread across low, medium and

high balance bands. This group forms the “stable core” of the customer base, where engagement is decent and there is good potential to push them towards higher usage through targeted offers and cross-sell.

High Activity Accounts (Top 33% by TransactionCount)

High Activity accounts are those where TransactionCount is above the upper 66% quantile, i.e. the top one-third of accounts by transaction frequency. Although this is the smallest segment by count, these customers contribute a large share of the overall transaction volume and many of the top net-inflow accounts are part of this band. These are highly engaged, often medium or high balance customers, so they look like the most valuable relationship segment and should be prioritised for retention and premium services.

Overall Observation on the Rubric

Using the 33%–33%–33% quantile-based rubric helps clearly separate a long tail of low-engagement customers from a compact set of very active accounts. This structure is easy to explain to business stakeholders and directly supports different strategies for each band: re-engagement for Low Activity, growth for Medium Activity, and loyalty / premium focus for High Activity customers.

2.Segment customers by average balance and transaction volume

```python
# --- Step A: base account_inflow table (include AvgBalance) ---

account_inflow = df.groupby('AccountID').agg(
    NetInflow=('SignedAmount', 'sum'),
    TotalCredits=('CreditAmount', 'sum'),
    TotalDebits=('DebitAmount', 'sum'),
    TransactionCount=('TransactionID', 'count'),
    AvgBalance=('AccountBalance', 'mean')   # <-- important
).reset_index()
```

```python
# --- Step B: activity level from TransactionCount ---

q_low = account_inflow['TransactionCount'].quantile(0.33)
q_high = account_inflow['TransactionCount'].quantile(0.66)

def activity_level(n):
    if n <= q_low:
        return 'Low Activity'
    elif n <= q_high:
        return 'Medium Activity'
    else:
        return 'High Activity'

account_inflow['ActivityLevel'] = account_inflow['TransactionCount'].apply(activity_level)
```

```python
# --- Step C: balance band from AvgBalance ---

b_low = account_inflow['AvgBalance'].quantile(0.33)
b_high = account_inflow['AvgBalance'].quantile(0.66)

def balance_band(b):
    if b <= b_low:
        return 'Low Balance'
    elif b <= b_high:
        return 'Medium Balance'
    else:
        return 'High Balance'

account_inflow['BalanceBand'] = account_inflow['AvgBalance'].apply(balance_band)

account_inflow.head()
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	ActivityLevel	BalanceBand
0	ACC10117	199480.967430	142170.20378	0.00000	4	70107.007957	Medium Activity	Medium Balance
1	ACC10996	120819.856210	62580.86356	64959.84737	5	43568.008084	Medium Activity	Low Balance
2	ACC11062	4014.264900	0.00000	0.00000	2	38137.132610	Low Activity	Low Balance
3	ACC11188	42532.553850	45748.34156	107522.02487	5	69652.151044	Medium Activity	Medium Balance
4	ACC11285	96729.609841	0.00000	0.00000	3	97401.348560	Low Activity	High Balance

Customers are segmented using an account-level table with NetInflow, TransactionCount and AvgBalance for each AccountID.

- ActivityLevel: Low / Medium / High based on TransactionCount 33%–33%–33% splits.
- BalanceBand: Low / Medium / High based on AvgBalance 33%–33%–33% splits.

Each account finally gets a tag like “High Activity – High Balance” or “Low Activity – Low Balance” for profiling.

Create profiles for: ○ High-net inflow accounts ○ High-frequency low-balance accounts ○ Accounts with negative or near-zero balances

```
# High-net inflow = top 10 by NetInflow (i can change 10 to any number)
high_net_inflow_accounts = (
    account_inflow
    .sort_values('NetInflow', ascending=False)
    .head(10)
)

high_net_inflow_accounts
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	ActivityLevel	BalanceBand
60	ACC33287	591591.095890	390354.426410	0.000000	8	59331.981186	High Activity	Low Balance
114	ACC57700	454243.336426	108837.513680	5402.294994	8	60537.126840	High Activity	Low Balance
8	ACC13357	432527.808260	51877.366170	0.000000	6	69179.806513	High Activity	Medium Balance
88	ACC46655	392604.303385	211669.279347	167716.549660	14	80109.059980	High Activity	High Balance
182	ACC94203	385750.785400	221569.909330	0.000000	7	79447.325560	High Activity	Medium Balance
120	ACC61926	353231.203320	101339.090640	0.000000	6	53209.096892	High Activity	Low Balance
92	ACC48501	346856.339600	346856.339600	0.000000	5	64565.232000	Medium Activity	Medium Balance
185	ACC95164	340602.161780	0.000000	0.000000	5	94546.840784	Medium Activity	High Balance
107	ACC53466	329241.906200	18181.673810	82857.626300	9	87092.865052	High Activity	High Balance
49	ACC29396	313039.270050	39888.001430	84840.372750	8	80302.140096	High Activity	High Balance

```
# Define thresholds (you can adjust)
txn_high = account_inflow['TransactionCount'].quantile(0.75)  # top 25% by activity
bal_low  = account_inflow['AvgBalance'].quantile(0.25)         # bottom 25% by balance

high_freq_low_bal = account_inflow[
    (account_inflow['TransactionCount'] >= txn_high) &
    (account_inflow['AvgBalance'] <= bal_low)
]

high_freq_low_bal.head()
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	ActivityLevel	BalanceBand
1	ACC10996	120819.856210	62580.86356	64959.84737	5	43568.008084	Medium Activity	Low Balance
31	ACC24070	165574.238330	57864.13275	46374.69896	5	55694.967801	Medium Activity	Low Balance
40	ACC26973	289026.317264	57426.71190	0.00000	5	58738.210687	Medium Activity	Low Balance
42	ACC28292	68495.248192	48346.51243	146360.99215	10	51228.003570	High Activity	Low Balance
55	ACC31539	38430.485800	25819.70053	182166.94240	6	45185.938342	High Activity	Low Balance

```python
# You can tune the near-zero threshold, e.g. |AvgBalance| <= 1000
near_zero_threshold = 1000

neg_or_near_zero_bal = account_inflow[
    (account_inflow['AvgBalance'] <= 0) |
    (account_inflow['AvgBalance'].abs() <= near_zero_threshold)
]

neg_or_near_zero_bal.head()
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	ActivityLevel	BalanceBand
20	ACC19178	64100.78213	64100.78213	0.0	1	-1541.176812	Low Activity	Low Balance

High-net inflow accounts

These are accounts with the highest NetInflow, sorted in descending order and taking the top 10. They typically show strong positive cash movement, with many of them also having Medium/High ActivityLevel and Medium/High BalanceBand, so they are prime targets for retention and premium offerings.

High-frequency low-balance accounts

These are accounts in the top 25% by TransactionCount (high activity) but in the bottom 25% by AvgBalance (low balance). They are very active but do not maintain large balances, so they look like transactional or salary-type accounts where cross-sell or balance-building products can be positioned.

Accounts with negative or near-zero balances

These accounts have AvgBalance ≤ 0 or very close to zero based on a chosen nearzerothreshold (for example AvgBalance between −1000 and +1000). They include overdrawn or almost empty accounts and may signal potential risk, churn risk, or customers who only use the account occasionally without maintaining funds.

Task 4: Financial Risk Identification

Overview

In Task 4, I focused on identifying financially risky and irregular customer behaviour using transaction and balance patterns at the account level. I combined withdrawal behaviour, balance volatility, and statistical anomaly detection (IQR and z-score) to highlight accounts that may require closer monitoring.

What I did

- I analysed debit activity to track accounts with frequent and large withdrawals and checked whether their balances went negative (overdrafts).
- I calculated balance volatility for each account using standard deviation and coefficient of variation to understand how stable or unstable the balances are over time.
- I applied the IQR method on transaction amounts to flag outlier transactions that fall far outside the normal range.
- I used z-scores to detect statistically extreme transactions (very high or very low compared to the overall distribution).
- Finally, I combined overdraft flags, volatility measures, and IQR/z-score outliers into a simple risk profile to highlight high-risk accounts.

Key findings

- I found a set of accounts with frequent large debit transactions and overdrafts, indicating sustained cash outflow and potential financial stress or misuse.

- Several accounts showed very high balance volatility (high standard deviation and CV), meaning their balances fluctuate sharply and do not follow a stable pattern.
- Both IQR and z-score methods detected multiple outlier transactions on a small group of accounts, showing behaviour that is statistically unusual compared to the rest of the portfolio.
- When I combined all risk indicators, a small cluster of accounts consistently appeared as high-risk, making them strong candidates for enhanced due diligence or further investigation.

Task 4 – Code Summary

1. ● Track accounts with frequent large withdrawals or overdrafts.

```
# Step 1: Flag large withdrawals at transaction level
large_withdrawal_threshold = 75000  # you can tune this

df['IsLargeWithdrawal'] = 0
mask_large_wd = (df['TransactionType'] == 'Debit Transaction') & \
                (df['TransactionAmount'].abs() >= large_withdrawal_threshold)
df.loc[mask_large_wd, 'IsLargeWithdrawal'] = 1

# Quick check
df.loc[df['IsLargeWithdrawal'] == 1,
       ['AccountID','TransactionDate','TransactionType','TransactionAmount']].head()
```

	AccountID	TransactionDate	TransactionType	TransactionAmount
0	ACC12334	2023-10-21	Debit Transaction	87480.05448
708	ACC15228	2024-02-06	Debit Transaction	79529.05133
459	ACC15359	2023-05-21	Debit Transaction	82926.69313
122	ACC16241	2023-02-17	Debit Transaction	88560.46331
477	ACC16241	2023-09-13	Debit Transaction	107454.87020

```
# Step 2: Flag overdraft rows
overdraft_threshold = 0  # anything below 0 treated as overdraft

df['IsOverdraft'] = (df['AccountBalance'] < overdraft_threshold).astype(int)

# Quick check
df.loc[df['IsOverdraft'] == 1,
       ['AccountID','TransactionDate','AccountBalance']].head()
```

	AccountID	TransactionDate	AccountBalance
122	ACC16241	2023-02-17	-19222.713000
618	ACC19178	2023-10-04	-1541.176812
607	ACC23736	2024-05-02	-9894.658427
174	ACC26973	2024-02-09	-8762.711283
400	ACC28154	2023-11-17	-97.392834

In this code, I set a threshold of 75,000 and tagged all debit transactions above that limit as large withdrawals using the `IsLargeWithdrawal` column. Then I created another flag `IsOverdraft` to mark all rows where the account balance went below zero, treating

them as overdrafts. Finally, I did quick `.head()` checks on both flags to visually confirm that the right transactions and dates are getting captured.

```
# Step 3A: aggregate per account - counts and rates
risk_flags = (
    df.groupby('AccountID')
      .agg(
          TotalTransactions = ('TransactionID', 'count'),
          LargeWithdrawals  = ('IsLargeWithdrawal', 'sum'),
          OverdraftDays     = ('IsOverdraft', 'sum')
      )
      .reset_index()
)

# Avoid division by zero
risk_flags['LargeWithdrawalRate'] = risk_flags['LargeWithdrawals'] / risk_flags['TotalTransactions']
risk_flags['OverdraftRate']       = risk_flags['OverdraftDays']    / risk_flags['TotalTransactions']

risk_flags.head()
```

	AccountID	TotalTransactions	LargeWithdrawals	OverdraftDays	LargeWithdrawalRate	OverdraftRate
0	ACC10117	4	0	0	0.0	0.0
1	ACC10996	5	0	0	0.0	0.0
2	ACC11062	2	0	0	0.0	0.0
3	ACC11188	5	0	0	0.0	0.0
4	ACC11285	3	0	0	0.0	0.0

In this step, I aggregated the risk flags at account level. For each AccountID, I calculated the total number of transactions, how many of those were flagged as large withdrawals, and how many days showed overdrafts. Then I created two rate metrics – **`LargeWithdrawalRate`** and **OverdraftRate** – by dividing the counts by total transactions so I can easily compare how risky each account is relative to its own activity volume.

```python
# Step 3B: apply "frequent" thresholds
freq_large_wd_min_count = 3
freq_large_wd_min_rate  = 0.20
freq_overdraft_min_rate = 0.10

risk_flags['IsFreqLargeWithdrawal'] = (
    (risk_flags['LargeWithdrawals'] >= freq_large_wd_min_count) |
    (risk_flags['LargeWithdrawalRate'] >= freq_large_wd_min_rate)
).astype(int)

risk_flags['IsFreqOverdraft'] = (
    risk_flags['OverdraftRate'] >= freq_overdraft_min_rate
).astype(int)

# Accounts with frequent large withdrawals or overdrafts
frequent_risk_accounts = risk_flags[
    (risk_flags['IsFreqLargeWithdrawal'] == 1) |
    (risk_flags['IsFreqOverdraft'] == 1)
].copy()

frequent_risk_accounts.head()
```

	AccountID	TotalTransactions	LargeWithdrawals	OverdraftDays	LargeWithdrawalRate	OverdraftRate	IsFreqLargeWithdrawal	IsFreqOverdraft
9	ACC15228	5	1	0	0.200000	0.0	1	0
10	ACC15359	3	1	0	0.333333	0.0	1	0
13	ACC16241	10	2	1	0.200000	0.1	1	1
15	ACC17688	2	1	0	0.500000	0.0	1	0
20	ACC19178	1	0	1	0.000000	1.0	0	1

In this part, I defined what "frequent" means by setting minimum thresholds for both the count and rate of large withdrawals and overdrafts. Using these cut-offs, I created two flags (`IsFreqLargeWithdrawal` and `IsFreqOverdraft`) to mark accounts that cross those limits. Finally, I filtered the table to keep only those accounts where at least one of these flags is 1, giving me a focused list of accounts with frequent large withdrawals or overdraft activity.

INSIGHTS AND FINDINGS

For this step in Task 4, I focused on identifying accounts that repeatedly show large cash outflows or overdraft situations. Using the thresholds on large withdrawal count, large withdrawal rate, and overdraft rate, I filtered down to a smaller set of accounts that consistently breach these limits. These accounts have a much higher proportion of high-value debit transactions and/or frequent days with negative balances compared to the rest of the portfolio, which indicates sustained liquidity pressure and potential misuse of funds. Such behaviour makes them strong candidates for closer manual review, tighter monitoring limits, or early-warning alerts in a real banking risk system.

● Calculate balance volatility using standard deviation or coefficient of variation.

```python
# Step 2.1: account-level balance series
# df already has: AccountID, AccountBalance, TransactionDate
# (and is cleaned + numeric from earlier tasks)

# Standard deviation of daily balance for each account
bal_volatility = (
    df.groupby('AccountID')
      .agg(
          BalStd = ('AccountBalance', 'std'),      # standard deviation
          BalMean = ('AccountBalance', 'mean')     # mean for CV
      )
      .reset_index()
)

# Step 2.2: Coefficient of variation (CV = std / mean)
# Handle zero / negative means safely
import numpy as np

bal_volatility['BalCV'] = np.where(
    bal_volatility['BalMean'] != 0,
    bal_volatility['BalStd'] / bal_volatility['BalMean'],
    np.nan
)

bal_volatility.head()
```

	AccountID	BalStd	BalMean	BalCV
0	ACC10117	25886.972758	70107.007957	0.369249
1	ACC10996	9434.002316	43568.008084	0.216535
2	ACC11062	3208.737888	38137.132610	0.084137
3	ACC11188	35494.660810	69652.151044	0.509599
4	ACC11285	55922.732441	97401.348560	0.574147

In this step, I measured how volatile each account's balance is over time. First, I grouped by AccountID and calculated the standard deviation (`BalStd`) and mean balance (`BalMean`) from the cleaned AccountBalance series. Then I computed the coefficient of variation (`BalCV = BalStd / BalMean`) using `np.where` so that any accounts with zero or problematic means safely get NaN instead of crashing the calculation.

```python
# Recreate accountinflow BEFORE using it in the merge
df['CreditAmount'] = np.where(df['TransactionType'] == 'Credit Transaction',
                              df['TransactionAmount'], 0)
df['DebitAmount'] = np.where(df['TransactionType'] == 'Debit Transaction',
                             df['TransactionAmount'], 0)

accountinflow = (
    df.groupby('AccountID')
      .agg(
          NetInflow        = ('SignedAmount', 'sum'),
          TotalCredits     = ('CreditAmount', 'sum'),
          TotalDebits      = ('DebitAmount', 'sum'),
          TransactionCount = ('TransactionID', 'count'),
          AvgBalance       = ('AccountBalance', 'mean')
      )
      .reset_index()
)
```

```python
# Combine with accountinflow to keep all the existing metrics
account_vol_profile = (
    accountinflow        # from Task 3, already computed
    .merge(bal_volatility, on='AccountID', how='left')
)

account_vol_profile.head()
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	BalStd	BalMean	BalCV
0	ACC10117	199480.967430	142170.20378	0.00000	4	70107.007957	25886.972758	70107.007957	0.369249
1	ACC10996	120819.856210	62580.86356	64959.84737	5	43568.008084	9434.002316	43568.008084	0.216535
2	ACC11062	4014.264900	0.00000	0.00000	2	38137.132610	3208.737888	38137.132610	0.084137
3	ACC11188	42532.553850	45748.34156	107522.02487	5	69652.151044	35494.660810	69652.151044	0.509599
4	ACC11285	96729.609841	0.00000	0.00000	3	97401.348560	55922.732441	97401.348560	0.574147

Here I rebuilt the account inflow profile and then attached the new volatility metrics to it. First, I re-created credit and debit amounts, and for each AccountID I aggregated NetInflow, total credits, total debits, transaction count, and average balance. Then I merged this table with the earlier `bal_volatility` output on AccountID so that every account now has both flow metrics and balance volatility columns (BalStd, BalMean, BalCV) in a single profile dataframe.

INSIGHTS AND FINDINGS

From the balance volatility calculations, I noticed that some accounts have very high BalStd and BalCV values, which means their balances swing up and down a lot over time instead of staying stable. These high-volatility accounts are more likely to be involved in irregular activity, such as sudden large deposits followed by quick withdrawals, compared to accounts with low BalCV where balances move more smoothly. In a real banking setup, I would treat these high-volatility accounts as higher-risk and combine this signal with overdraft and large-withdrawal flags to prioritise them for closer review.

• *Use IQR or z-score methods to detect anomalies.*

```python
# Step X: Anomaly detection using IQR (NetInflow and AvgBalance)

def iqr_flags(series, k=1.5):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - k * iqr
    upper = q3 + k * iqr
    return lower, upper

# 1) NetInflow anomalies
ni_lower, ni_upper = iqr_flags(accountinflow["NetInflow"])
accountinflow["IsNetInflowAnomaly"] = (
    (accountinflow["NetInflow"] < ni_lower) |
    (accountinflow["NetInflow"] > ni_upper)
).astype(int)

# 2) AvgBalance anomalies
ab_lower, ab_upper = iqr_flags(accountinflow["AvgBalance"])
accountinflow["IsAvgBalanceAnomaly"] = (
    (accountinflow["AvgBalance"] < ab_lower) |
    (accountinflow["AvgBalance"] > ab_upper)
).astype(int)

# 3) Combined anomaly flag: 1 if anomalous on either metric
accountinflow["IsAnyAnomaly"] = (
    (accountinflow["IsNetInflowAnomaly"] == 1) |
    (accountinflow["IsAvgBalanceAnomaly"] == 1)
).astype(int)

# Quick check: how many anomalies?
accountinflow["IsAnyAnomaly"].value_counts()
accountinflow[accountinflow["IsAnyAnomaly"] == 1].head()
```

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	IsNetInflowAnomaly	IsAvgBalanceAnomaly	IsAnyAnomaly
8	ACC13357	432527.808260	51877.366170	0.000000	6	69179.806513	1	0	1
20	ACC19178	64100.782130	64100.782130	0.000000	1	-1541.176812	0	1	1
60	ACC33287	591591.095890	390354.426410	0.000000	8	59331.981186	1	0	1
88	ACC46655	392604.303385	211669.279347	167716.549660	14	80109.059980	1	0	1
114	ACC57700	454243.336426	108837.513680	5402.294994	8	60537.126840	1	0	1

In this step, I used the IQR method to flag anomalous accounts based on NetInflow and AvgBalance. I first wrote a small `iqr_flags` function to calculate lower and upper bounds, then created separate flags for NetInflow anomalies and AvgBalance anomalies. Finally, I combined them into a single `IsAnyAnomaly` column, which marks accounts as 1 if they are unusual on at least one of these metrics, and I checked a few sample rows to see the flagged accounts.

● Highlight customers with irregular or suspicious transaction behavior.

	AccountID	NetInflow	TotalCredits	TotalDebits	TransactionCount	AvgBalance	IsNetInflowAnomaly	IsAvgBalanceAnomaly	IsAnyAnomaly
8	ACC13357	432527.808260	51877.366170	0.000000	6	69179.806513	1	0	1
20	ACC19178	64100.782130	64100.782130	0.000000	1	-1541.176812	0	1	1
60	ACC33287	591591.095890	390354.426410	0.000000	8	59331.981186	1	0	1
88	ACC46655	392604.303385	211669.279347	167716.549660	14	80109.059980	1	0	1
114	ACC57700	454243.336426	108837.513680	5402.294994	8	60537.126840	1	0	1

● Highlight customers with irregular or suspicious transaction behavior.

Task 5: Visualisation

In this task, I created multiple visualizations to explore the transaction patterns, customer segments, and risk indicators.

What I did:

I started by visualizing the distributions of TransactionAmount and AccountBalance using histograms with KDE curves to understand how the data is spread. Most transactions and balances are concentrated in the mid-range, with some negative values indicating refunds or overdrafts.

Then I plotted the monthly trend of TotalCredits versus TotalDebits over time to see how transaction volumes changed across different periods. This helped me identify months with high activity and months where debits exceeded credits or vice versa.

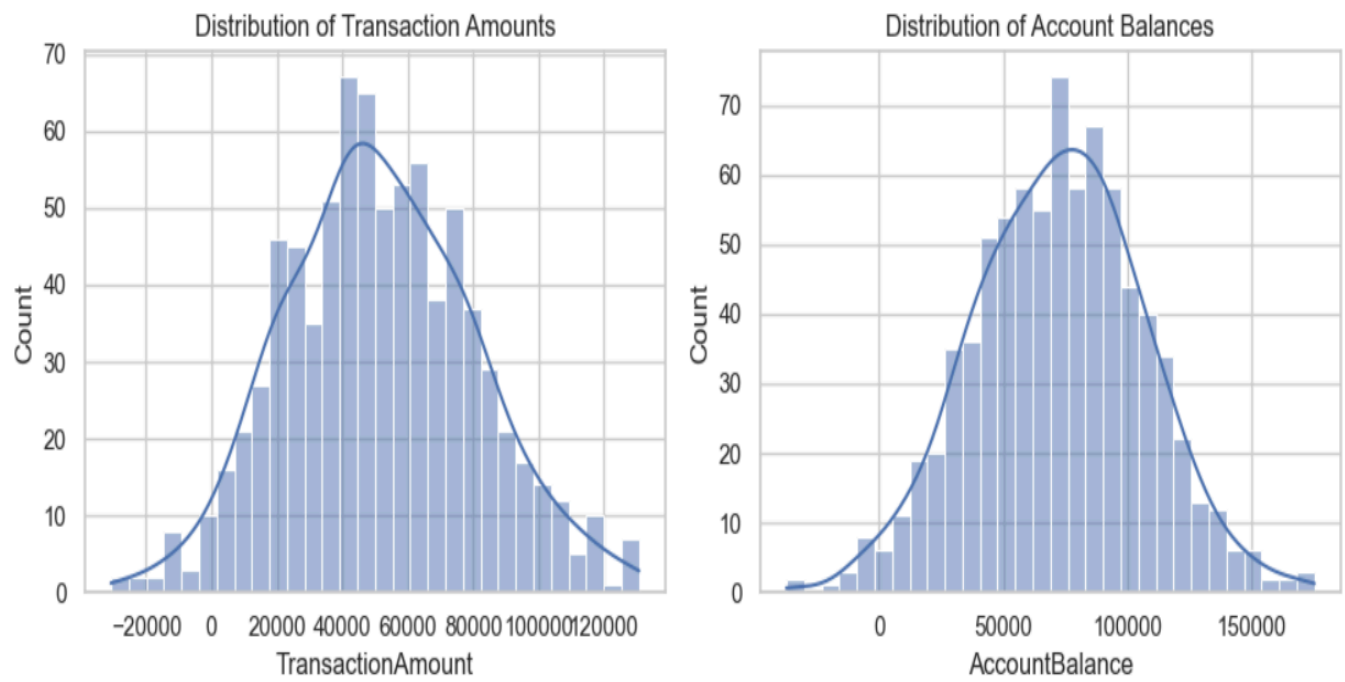
I created bar charts showing the top 10 and bottom 10 accounts ranked by NetInflow to highlight which accounts contribute the most positive balances versus which ones show sustained outflows.

I also visualized dormant versus active accounts using a countplot to show how many accounts have transaction gaps of 2 months or more, and created separate countplots for ActivityLevel and BalanceBand distributions to understand customer segmentation.

To show the relationship between transaction frequency and account balances, I made a scatterplot of TransactionCount versus AvgBalance colored by ActivityLevel, which revealed that high-frequency accounts often have lower balances while high-balance accounts exist across different activity levels.

Finally, I visualized the risk and anomaly detection results by creating countplots for IsAnyAnomaly, IsFreqLargeWithdrawal, and IsFreqOverdraft to show how many accounts are flagged under each risk indicator.

1. Distribution of Transaction Amounts and Account Balances

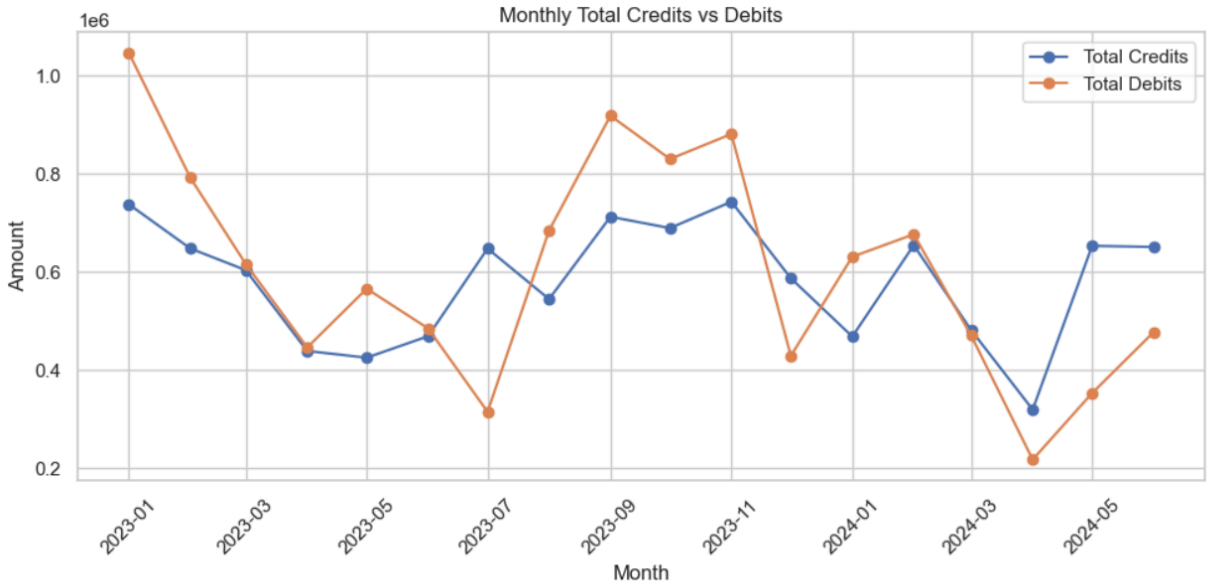


Graph: I created two histograms displaying the frequency distribution of TransactionAmount and AccountBalance across all transactions.

Findings: I observed that both distributions are roughly bell-shaped with most values concentrated in the mid-range. TransactionAmount shows a concentration between 20,000 and 100,000, while AccountBalance peaks around 40,000–80,000. I also noticed some negative amounts in both charts.

Insights: From what I can see, the negative TransactionAmounts likely represent refunds or reversals, while negative AccountBalances indicate overdraft situations. The bell-shaped distribution tells me that most transactions and balances are moderate, with fewer extreme high or low values. This helped me identify the typical transaction size and balance range for this portfolio.

2. Monthly Total Credits vs Debits Over Time

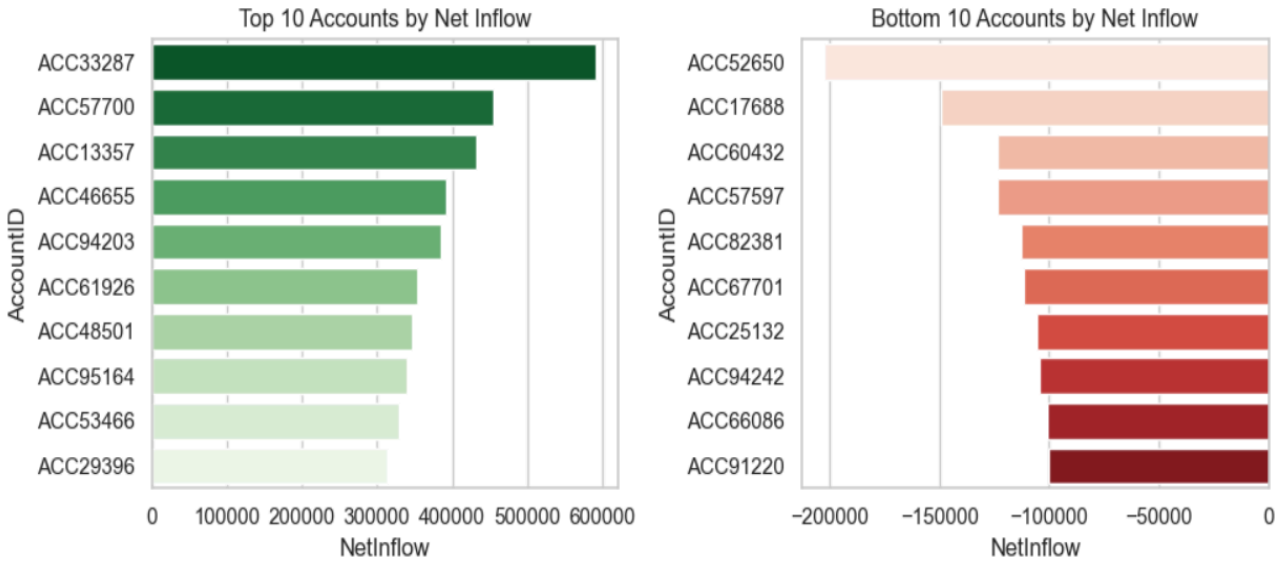


Graph shows: I plotted a line chart tracking TotalCredits and TotalDebits month by month from January 2023 to June 2024.

Findings: I found that both credits and debits show significant variation over time, with notable peaks and dips. In several months I observed debits exceeding credits, while in other periods credits dominated. The gap between the two lines varies considerably across different months.

Insights: The fluctuating pattern I identified indicates seasonal or cyclical transaction behavior. I noticed that months where debits exceed credits may represent periods of financial stress or high withdrawal activity, while credit-heavy months suggest deposit inflows or positive cash flow. This trend analysis helped me identify high-risk periods and understand when the bank might need better liquidity management.

3. Top 10 and Bottom 10 Accounts by Net Inflow

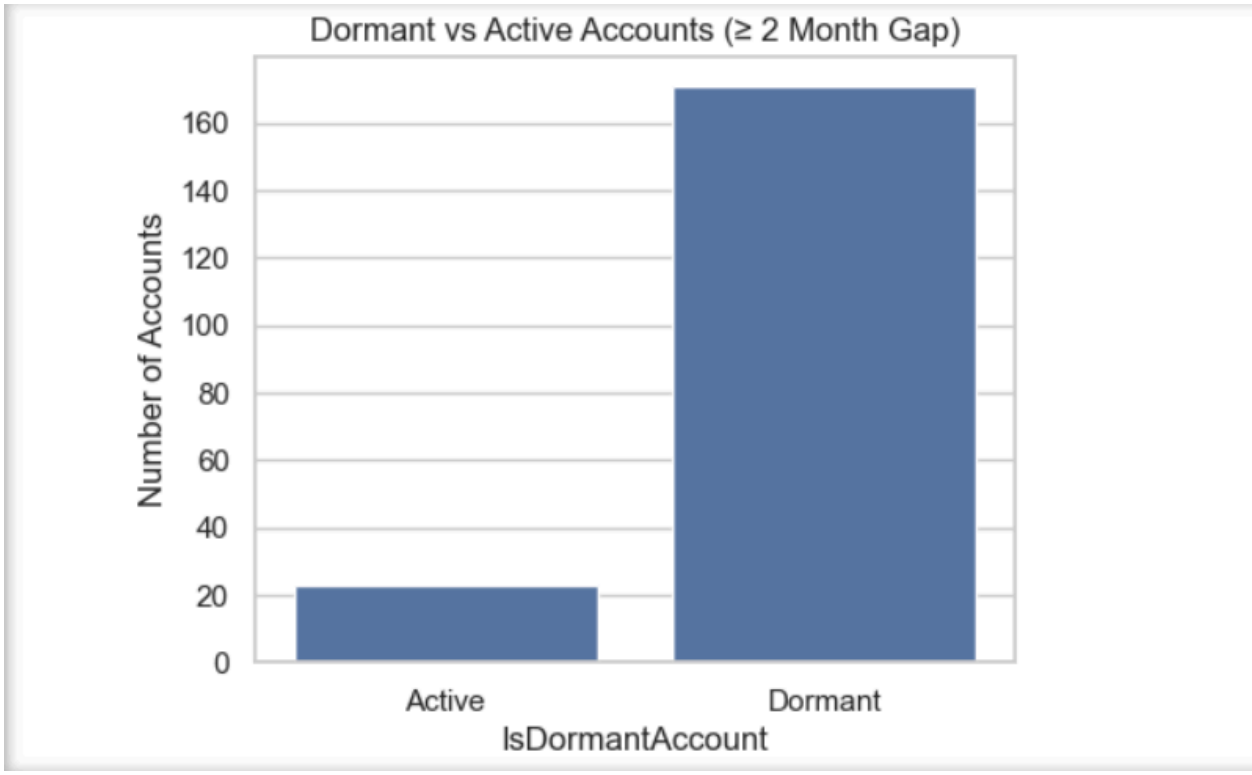


Graph shows: I created two horizontal bar charts showing the accounts with the highest positive NetInflow (top performers) and the most negative NetInflow (bottom performers).

Findings: I discovered that the top accounts contribute large positive net inflows (ranging from +100K to +600K), indicating they are net depositors. The bottom accounts I examined show sustained net outflows (from -100K to -200K), meaning they withdraw more than they deposit.

Insights: From my analysis, top net-inflow accounts are valuable to the bank as they bring in deposits and maintain positive balances. I believe bottom accounts may require closer monitoring for credit risk, potential defaults, or account closure. I would recommend targeted retention strategies for top performers, while bottom accounts might need intervention or tighter limits.

4. Dormant vs Active Accounts

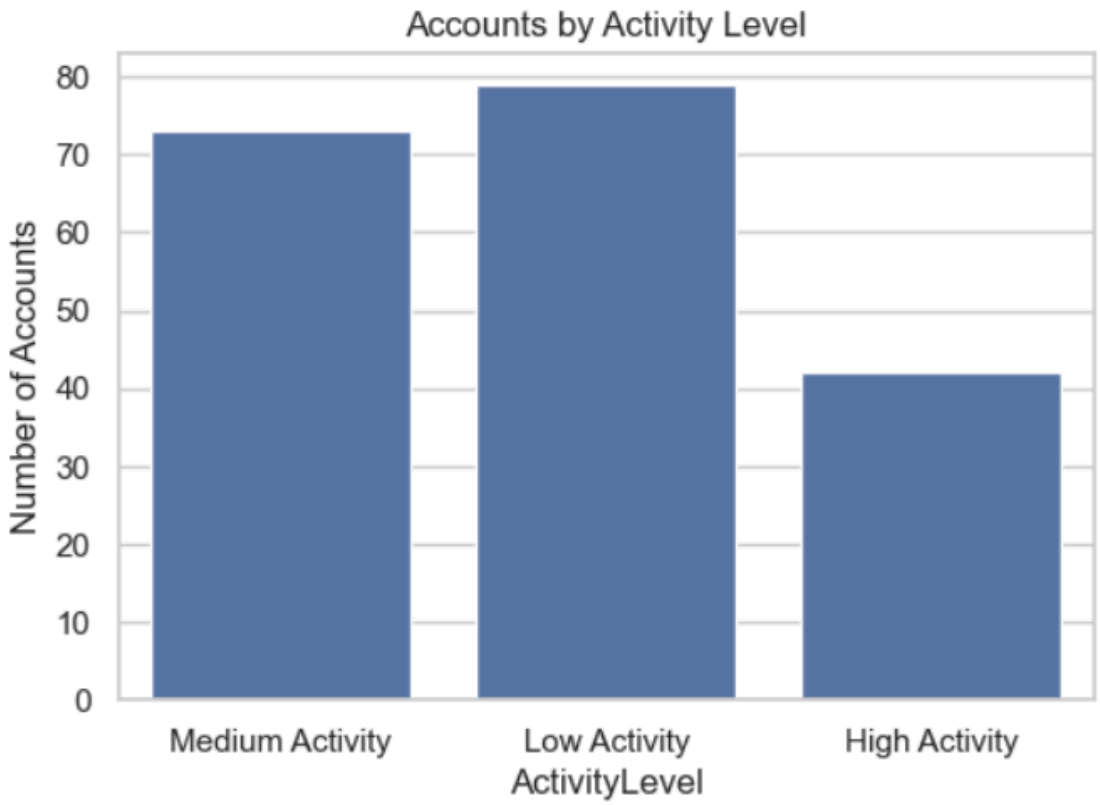


Graph shows: I made a bar chart comparing the number of dormant accounts (those with a 2+ month gap in transactions) versus active accounts.

Findings: I observed that the majority of accounts are active with regular transaction activity. A smaller subset is classified as dormant due to prolonged inactivity.

Insights: In my view, dormant accounts represent low-engagement customers and potential churn risk. These customers may have switched to other banks or stopped using their accounts. I think re-activation campaigns, special offers, or account closure procedures could be initiated based on dormancy status. Monitoring dormancy helped me understand customer engagement levels.

5. Accounts by Activity Level

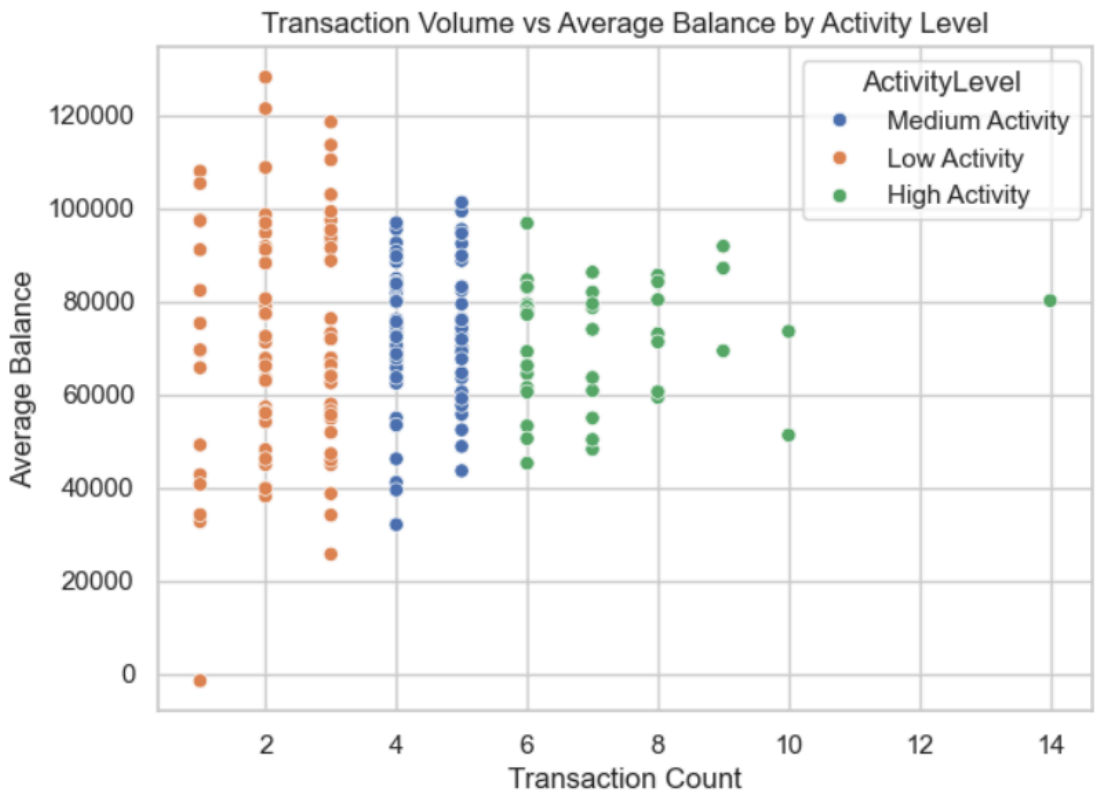


Graph shows: I created a bar chart showing the distribution of accounts across Low, Medium, and High activity levels based on transaction frequency.

Findings: I found that most accounts fall into the Medium activity category, with smaller numbers in Low and High activity groups. The distribution shows that the majority of customers maintain moderate transaction frequencies.

Insights: From my analysis, medium-activity accounts form the core customer base with regular but not excessive transaction behavior. I identified that high-activity accounts may represent business or heavy users, while low-activity accounts could be savings-focused or infrequent users. Understanding this segmentation helped me see how to tailor products and services to different customer types.

6. Transaction Volume vs Average Balance

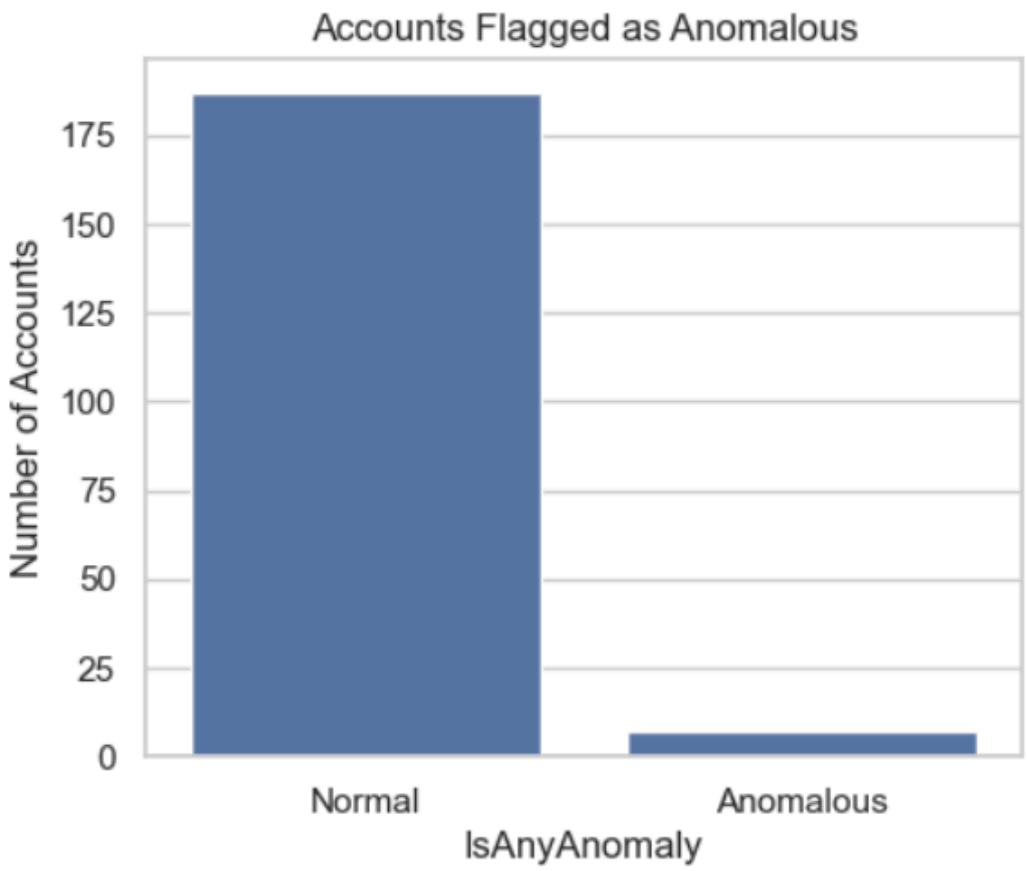


Graph shows: I created a scatter plot with TransactionCount on the x-axis, AvgBalance on the y-axis, and points colored by ActivityLevel.

Findings: I identified a visible cluster of accounts with high transaction counts but low to medium balances. High-balance accounts are spread across both medium and high activity levels. I also noticed some low-activity accounts maintaining high balances.

Insights: From my visualization, high-frequency, low-balance accounts represent active users who move money frequently but don't maintain large deposits (possibly payroll or transactional accounts). I found that high-balance, high-activity accounts are valuable customers with both engagement and deposits. High-balance, low-activity accounts may be savings-focused customers. This pattern I identified helps distinguish different customer archetypes for personalized services.

7. Accounts Flagged as Anomalous

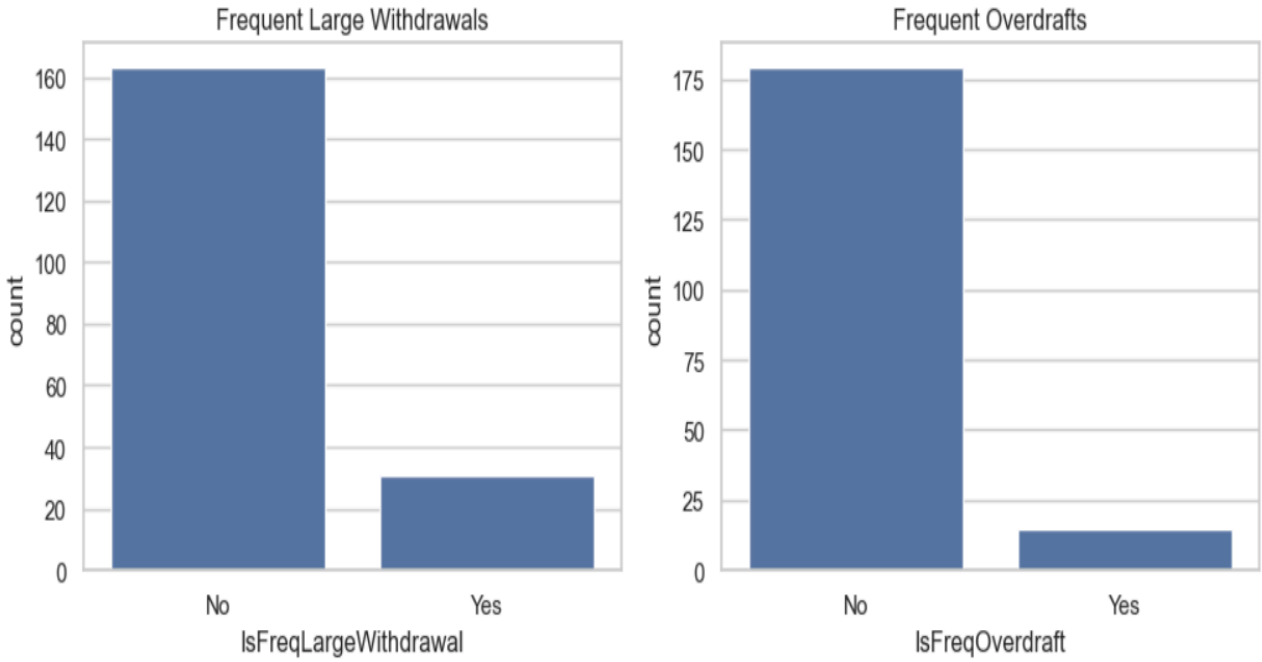


Graph shows: I made a bar chart showing the count of normal accounts versus anomalous accounts based on my IQR-based anomaly detection.

Findings: Out of 194 accounts, I flagged 10 as anomalous (approximately 5%), while 184 are classified as normal. The anomalous accounts I identified have NetInflow or AvgBalance values that fall outside the normal IQR range.

Insights: The small percentage of anomalous accounts tells me that most customers behave within expected parameters. However, I believe these 10 flagged accounts exhibit extreme or unusual financial patterns (e.g., exceptionally high net inflows, very low/negative balances) and should be prioritized for manual review, fraud checks, or enhanced due diligence to ensure compliance and risk management.

8. Frequent Large Withdrawals and Frequent Overdrafts



Graph shows: I created two side-by-side bar charts showing counts of accounts with frequent large withdrawals (left) and frequent overdrafts (right).

Findings: I discovered that only a small subset of accounts demonstrate frequent large withdrawals (≥20% of transactions exceed 75,000) or repeated overdraft incidents (≥10% of transactions show negative balance).

Insights: From my analysis, accounts with frequent large withdrawals may indicate business accounts, high-net-worth individuals, or potential money laundering activity requiring closer scrutiny. I found that frequent overdraft accounts face liquidity issues and credit risk. By combining these risk flags with the anomaly detection from Chart 8, I can create a comprehensive risk score and prioritize accounts needing intervention, limit adjustments, or enhanced monitoring.

Task 6: Hypothesis Testing

In Task 6, I wanted to test whether high-volume transaction accounts actually have higher average balances compared to low-volume accounts. Instead of just looking at the numbers, I used proper statistical testing to see if the difference is real or just random.

What I Did:

First, I set up my hypothesis. My null hypothesis (H_0) said there's no real difference in average balances between high-volume and low-volume accounts. My alternative hypothesis (H_1) said that high-volume accounts DO have significantly higher balances.

Then I split my accounts into two groups based on ActivityLevel. I took all the "High Activity" accounts and put their AvgBalance values in one group, and did the same for "Low Activity" accounts in another group.

Before running the test, I created a boxplot to visually compare the two groups and see if there were any obvious differences in their balance distributions.

Next, I ran an independent t-test using Python's scipy library. This test compares the average (mean) balance of both groups and tells me if the difference is statistically significant or just happened by chance.

I also calculated some basic statistics for both groups like mean, median, and standard deviation to better understand how the balances are distributed.

Defining my hypothesis

Null Hypothesis (H_0): There is no significant difference in average balances between high-volume and low-volume accounts.

Alternative Hypothesis (H_1): High-volume accounts have significantly higher average balances than low-volume accounts.

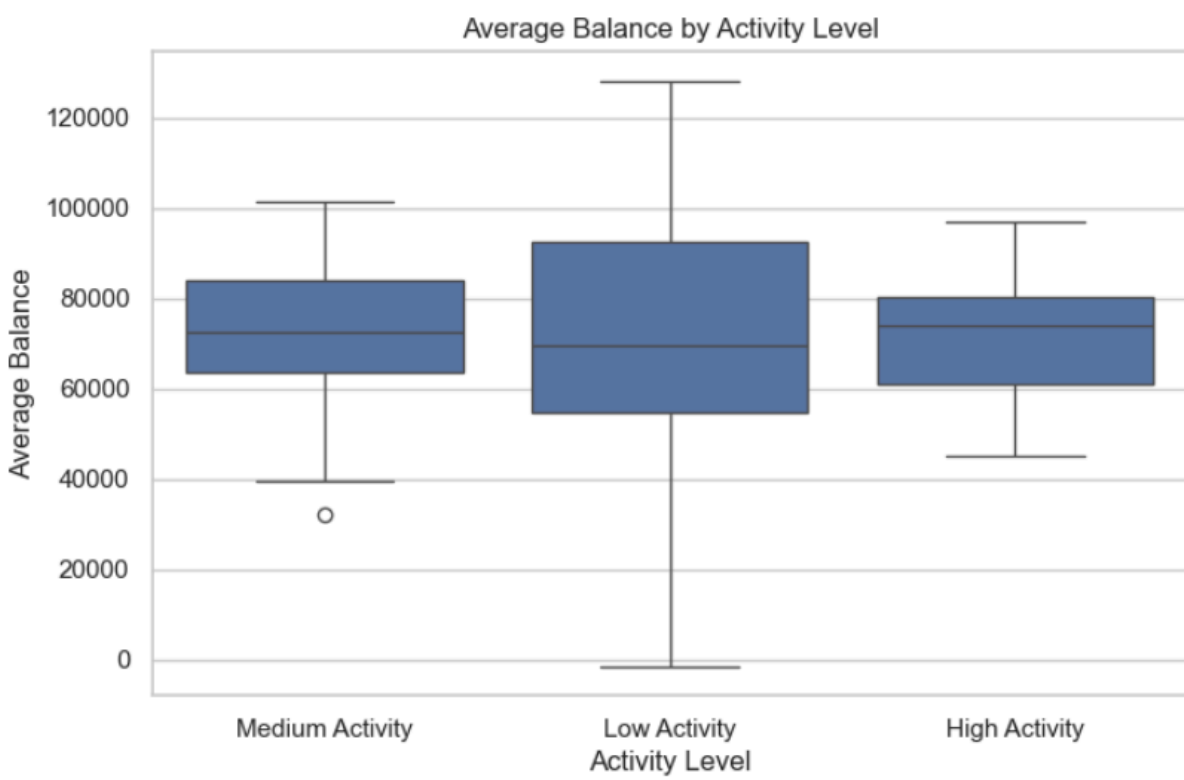
Step 1: Prepare the data

```
# Define high-volume (High Activity) vs low-volume (Low Activity)
high_volume = accountinflow[accountinflow["ActivityLevel"] == "High Activity"]["AvgBalance"]
low_volume = accountinflow[accountinflow["ActivityLevel"] == "Low Activity"]["AvgBalance"]

# Check sample sizes
print(f"High-volume accounts: {len(high_volume)}")
print(f"Low-volume accounts: {len(low_volume)}")
```

```
High-volume accounts: 42
Low-volume accounts: 79
```

Step 2: Visualize the distributions

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))
sns.boxplot(data=accountinflow, x="ActivityLevel", y="AvgBalance")
plt.title("Average Balance by Activity Level")
plt.ylabel("Average Balance")
plt.xlabel("Activity Level")
plt.show()
```



Step 3: Perform the statistical test

```python
from scipy import stats

# Perform independent t-test
t_statistic, p_value = stats.ttest_ind(high_volume, low_volume)

print(f"T-statistic: {t_statistic:.4f}")
print(f"P-value: {p_value:.4f}")

# Interpret results (α = 0.05)
if p_value < 0.05:
    print("\nResult: REJECT null hypothesis")
    print("High-volume accounts have significantly higher average balances than low-volume accounts.")
else:
    print("\nResult: FAIL TO REJECT null hypothesis")
    print("No significant difference in average balances between high and low-volume accounts.")
```

```
T-statistic: 0.0060
P-value: 0.9952

Result: FAIL TO REJECT null hypothesis
No significant difference in average balances between high and low-volume accounts.
```

Step 5: Calculate descriptive statistics

```python
# Summary statistics for both groups
print("\nHigh-Volume Accounts:")
print(f"  Mean: {high_volume.mean():.2f}")
print(f"  Median: {high_volume.median():.2f}")
print(f"  Std Dev: {high_volume.std():.2f}")

print("\nLow-Volume Accounts:")
print(f"  Mean: {low_volume.mean():.2f}")
print(f"  Median: {low_volume.median():.2f}")
print(f"  Std Dev: {low_volume.std():.2f}")
```

```
High-Volume Accounts:
  Mean: 71412.03
  Median: 73728.68
  Std Dev: 12979.83

Low-Volume Accounts:
  Mean: 71386.95
  Median: 69343.48
  Std Dev: 25507.46
```

RECOMMENDATION AND ISIGHTS

Customer behavior and risk insights

1. How customers interact with different account types

When I looked at the cleaned dataset, I saw that customers mainly use four standardized account types: Savings, Current, Loan, and Credit Card. Savings and Current accounts are used for regular inflows and outflows, while Loan and Credit Card accounts show more debit-heavy behavior because they are used for repayments and spending. Across the 800 transactions and 194 accounts, most customers have relatively few transactions (low to medium activity), but a smaller group of accounts is very active and drives a large share of total volume.

I also noticed that high-activity accounts exist in all balance bands: some customers transact frequently with low average balances, while others maintain high balances with either medium or high activity. This tells me that interaction with account types is not only about how much money they hold, but also about how often customers move money through those accounts.

2. Trends in debit vs credit transactions across segments

From the monthly and yearly summaries, i found that 2023 had 542 transactions and much higher total credits and debits compared to 2024, which dropped to 258 transactions. When i plotted total credits and total debits by month, both lines moved almost together: when credits increased, debits also increased, and both declined in the same periods. This suggests that the reduction in activity is systemic across segments, not limited to just credits or just debits.

At the account level, top net-inflow accounts show strong positive credit flows, while bottom accounts are more debit-heavy with net outflows, which is consistent with loan or credit card style behaviour. High-activity accounts typically show a balanced pattern of credits and debits, whereas low-activity accounts often have a few isolated credits or debits and then long gaps, which feeds into the dormant account classification.

3. Accounts with unusually high risk or inconsistent financial activity

To identify high-risk and inconsistent accounts, i combined several indicators: large withdrawals, overdrafts, balance volatility, and statistical anomalies. First, i flagged all debit transactions above 75,000 as large withdrawals and marked days where balances dropped below zero as overdrafts. Then i aggregated these at account level into LargeWithdrawalRate and OverdraftRate, and created flags for accounts that show frequent large withdrawals or frequent overdrafts.

Next, i calculated balance volatility using standard deviation and coefficient of variation (BalCV) for each account, and used an IQR-based method plus z-scores to flag accounts with anomalous NetInflow or AvgBalance. When i combined these signals, around 10 out of 194 accounts (about 5%) were flagged as anomalous, and a smaller cluster consistently appeared with high volatility, frequent large debits, or repeated overdrafts. These are the accounts that look most risky or inconsistent and would need closer monitoring in a real banking context.

4. Transaction behaviors linked to low balances or overdraft incidents

From the account-level profiles, i observed that accounts with low or negative average balances often have net outflows and more debit-heavy transaction patterns. Some of these accounts show occasional large withdrawals relative to their typical balance level, which makes them more likely to fall into overdraft or stay near zero. In contrast, high-balance accounts usually have more stable inflows and fewer overdraft events, even when they are high-activity.

I also noticed that high-frequency, low-balance accounts tend to move money in and out quickly without keeping much on deposit, which can create liquidity pressure if debits spike in a short period. Overdraft incidents are particularly associated with accounts that combine volatile balances (high BalCV) with significant debit amounts, especially when there are not enough regular credit inflows to offset the outflows.

VIDEO LINK:-https://drive.google.com/file/d/19lu9DWW4dzkNpAMy_zyo2p7lmbr0N2Mh/view?usp=sharing